

Experiment ‘p_6_4_sum_without_smallest_array’ Results

June 4, 2026

Experiment outcome: SUCCESS

Bad responses: 0

Responses containing assume : 0

Responses containing expect : 0

Resolution attempts: 1

Hard fails (resolution): 0

Soft fails (resolution): 0

Verification attempts: 1

Problem Specification

Problem name: p_6_4_sum_without_smallest_array

Natural language statement: Write a method that computes the sum of an array of values, except for the smallest one, in a single loop. In the loop, update the sum and the smallest value. After the loop, return the difference.

Method signature: p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)

Ensures

- `sum == sum_array(inputs[..]) - find_smallest(inputs[..])`

Requires

- `inputs.Length >= 1`

Functional Code Given

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
  }
}
```

```
function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

GenAI interactions

Below you will find all interactions between the ‘user’ (program) and the ‘assistant’ (OpenAI).

Program → GenAI

You are given the following task to perform in Dafny:

Write a method that computes the sum of an array of values, except for the smallest one,
→ in a single loop. In the loop, update the sum and the smallest value. After the
→ loop, return the difference.

The signature should be:

`p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)`

The method should respect the following contract:

`requires inputs.Length >= 1, ensures sum == sum_array(inputs[..]) - find_smallest(inputs
→ [...])`

The contract uses the following dafny code:

```
function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
}

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}
```

These function(s) must not be used in your implementation of the method.

Produce and show only the Dafny body of this method, including the curly braces that
→ surround it. Do not show the signature nor contract. You must not use 'assume' nor
→ 'expect' anywhere in your code.

GenAI → Program

System fingerprint: null

ID: chatcmpl-Dn1XsphJxUqBjD7MDG4vaxaUzckm6

Model: gpt-5.5-2026-04-23

Created at: 1780576384

Finish reason: stop

Usage: 762 tokens in, and 4064 tokens out

```
{
  var i := inputs.Length - 1;
  var total := inputs[i];
  var smallest := inputs[i];

  assert 0 <= i < inputs.Length;
  assert |inputs[i..]| == 1;
  assert inputs[i..][0] == inputs[i];
  assert sum_array(inputs[i..]) == inputs[i];
  assert find_smallest(inputs[i..]) == inputs[i];

  while i > 0
    invariant 0 <= i < inputs.Length
    invariant total == sum_array(inputs[i..])
    invariant smallest == find_smallest(inputs[i..])
  {
    var j := i - 1;
    assert 0 <= j < inputs.Length;
    assert i == j + 1;
    assert |inputs[j..]| >= 2;
    assert inputs[j..][0] == inputs[j];
    assert inputs[j..][1..] == inputs[i..];

    assert sum_array(inputs[j..]) == inputs[j] + sum_array(inputs[i..]);
    assert find_smallest(inputs[j..]) ==
      (if inputs[j] <= find_smallest(inputs[i..]) then inputs[j] else find_smallest(inputs
        ↪ [i..]));

    total := inputs[j] + total;
    assert total == sum_array(inputs[j..]);

    if inputs[j] <= smallest {
      assert inputs[j] <= find_smallest(inputs[i..]);
      smallest := inputs[j];
    } else {
      assert !(inputs[j] <= find_smallest(inputs[i..]));
    }
    assert smallest == find_smallest(inputs[j..]);

    i := j;
  }

  assert i == 0;
```

```

    assert inputs[i..] == inputs[..];
    assert total == sum_array(inputs[..]);
    assert smallest == find_smallest(inputs[..]);
    sum := total - smallest;
}

```

Final Program

```

function find_smallest(inputs: seq<int>): (ret: int)
  requires |inputs| >= 1
  ensures forall x :: x in inputs ==> ret <= x
{
  if |inputs| == 1 then inputs[0]
  else
    assert 2 <= |inputs|;
    var rest_smallest := find_smallest(inputs[1..]);
    var ret: int := if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    assert inputs == [inputs[0]] + inputs[1..];
    assert ret == if inputs[0] <= rest_smallest then inputs[0] else rest_smallest;
    ret
  }

function sum_array(inputs: seq<int>): int
{
  if |inputs| == 0 then 0
  else inputs[0] + sum_array(inputs[1..])
}

method p_6_4_sum_without_smallest_array(inputs: array<int>) returns (sum: int)
  requires inputs.Length >= 1
  ensures sum == sum_array(inputs[..]) - find_smallest(inputs[..])
{
  var i := inputs.Length - 1;
  var total := inputs[i];
  var smallest := inputs[i];

  assert 0 <= i < inputs.Length;
  assert |inputs[i..]| == 1;
  assert inputs[i..][0] == inputs[i];
  assert sum_array(inputs[i..]) == inputs[i];
  assert find_smallest(inputs[i..]) == inputs[i];

  while i > 0
    invariant 0 <= i < inputs.Length
    invariant total == sum_array(inputs[i..])
    invariant smallest == find_smallest(inputs[i..])
  {
    var j := i - 1;
    assert 0 <= j < inputs.Length;
    assert i == j + 1;

```

```

assert |inputs[j..]| >= 2;
assert inputs[j..][0] == inputs[j];
assert inputs[j..][1..] == inputs[i..];

assert sum_array(inputs[j..]) == inputs[j] + sum_array(inputs[i..]);
assert find_smallest(inputs[j..]) ==
  (if inputs[j] <= find_smallest(inputs[i..]) then inputs[j] else find_smallest(inputs
    ↪ [i..]));

total := inputs[j] + total;
assert total == sum_array(inputs[j..]);

if inputs[j] <= smallest {
  assert inputs[j] <= find_smallest(inputs[i..]);
  smallest := inputs[j];
} else {
  assert !(inputs[j] <= find_smallest(inputs[i..]));
}
assert smallest == find_smallest(inputs[j..]);

i := j;
}

assert i == 0;
assert inputs[i..] == inputs[..];
assert total == sum_array(inputs[..]);
assert smallest == find_smallest(inputs[..]);
sum := total - smallest;
}

```

Total Token Usage

Input tokens: 762

Output tokens: 4064

Reasoning tokens: 3574

Sum of 'total tokens': 4826

Experiment Timings

Overall Experiment started at 1780576383607, ended at 1780576475976, lasting 92369ms (92.37 seconds)

Iteration #1 started at 1780576383622, ended at 1780576475976, lasting 92354ms (92.35 seconds)